Docker & Containers – Running Your First Container

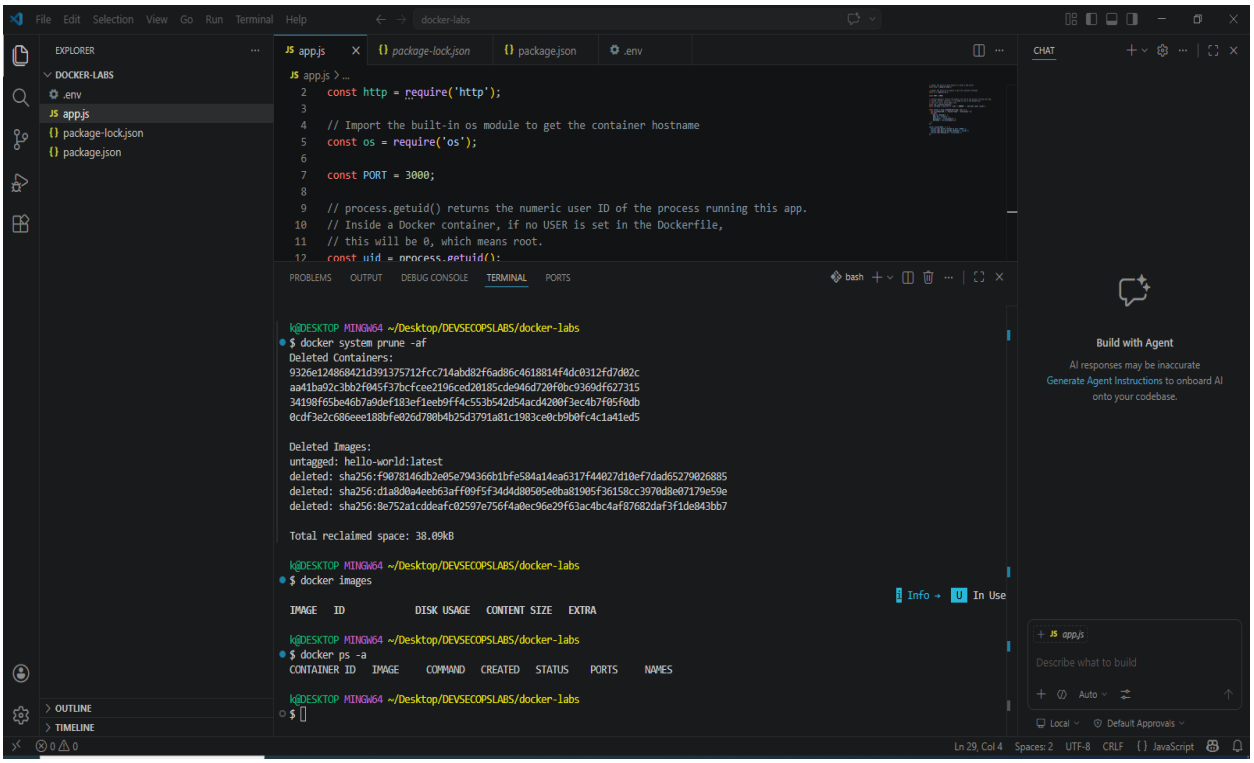
Detailed instructions and documentation are available in the GitHub repository:
https://github.com/samuel-nartey/devops-labs/blob/main/Docker%20%26%20Containers/Running%20Your%20First%20Container/README.md

Scenario 1: The Insecure Default

Objectives

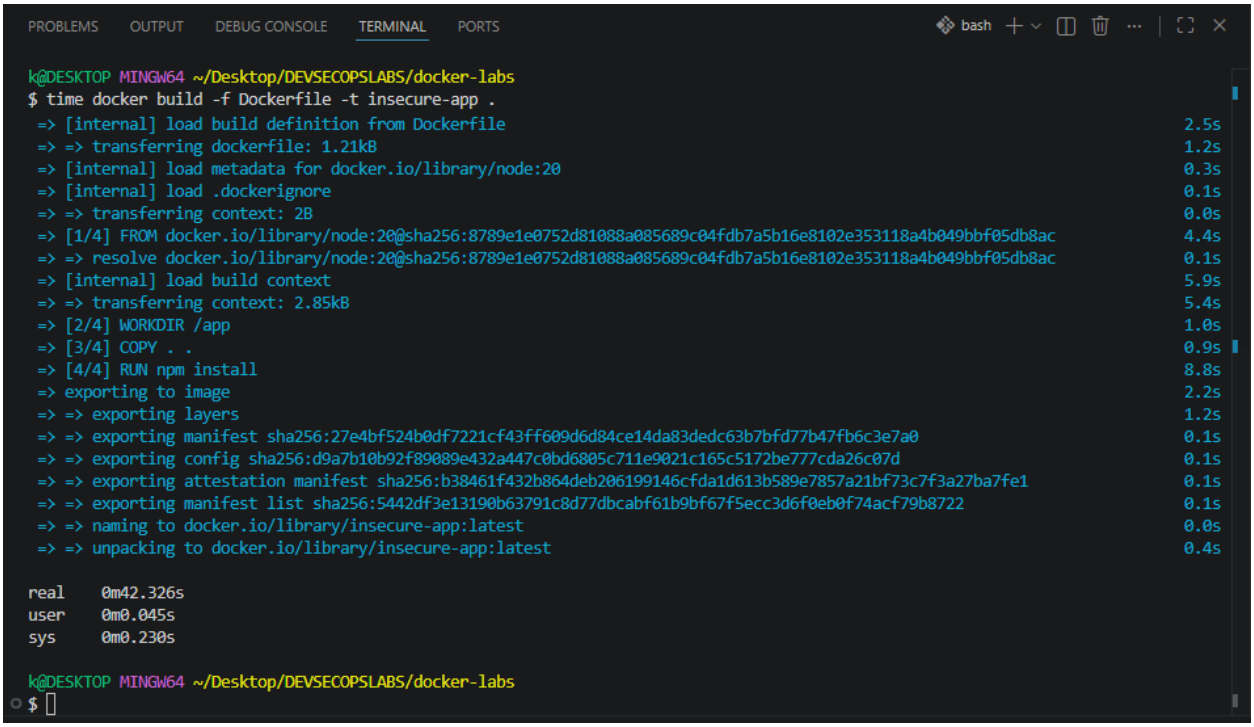
Understand what a basic Dockerfile looks like, why it is insecure by default, and how Docker layer caching works in practice using timed builds**.**

Steps Performed

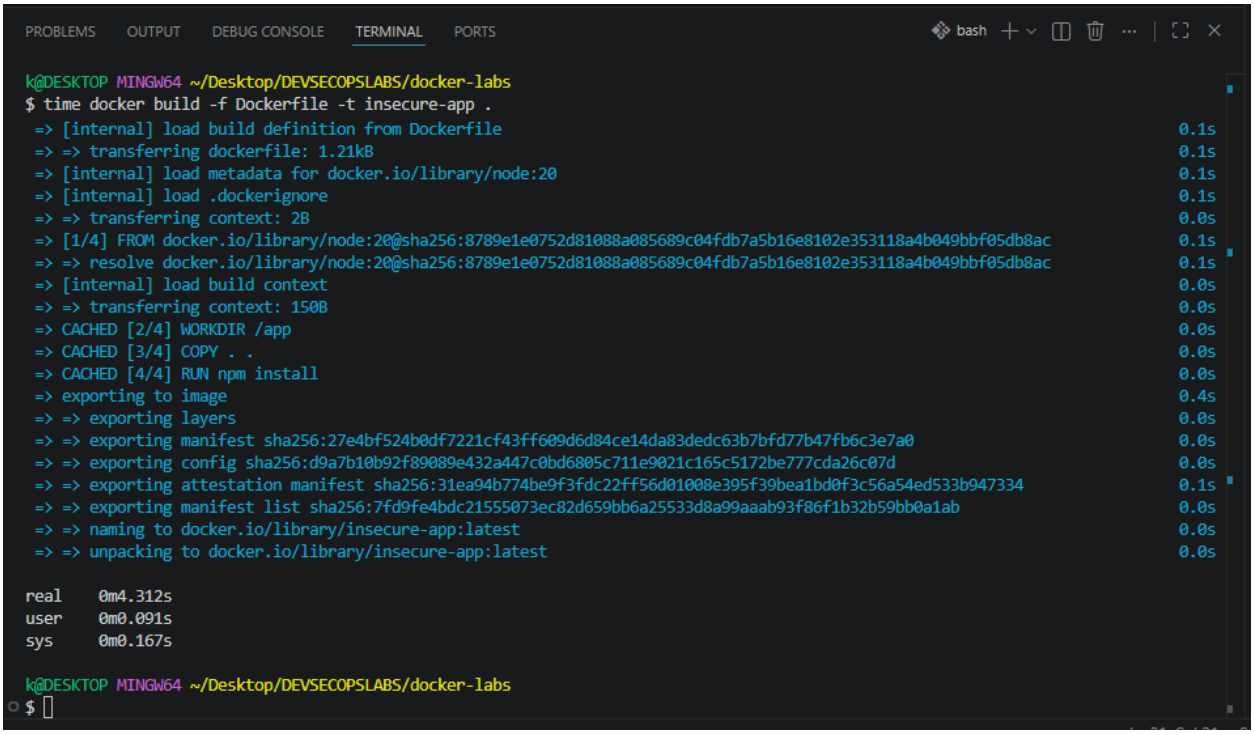


Layer Caching with Timed Builds

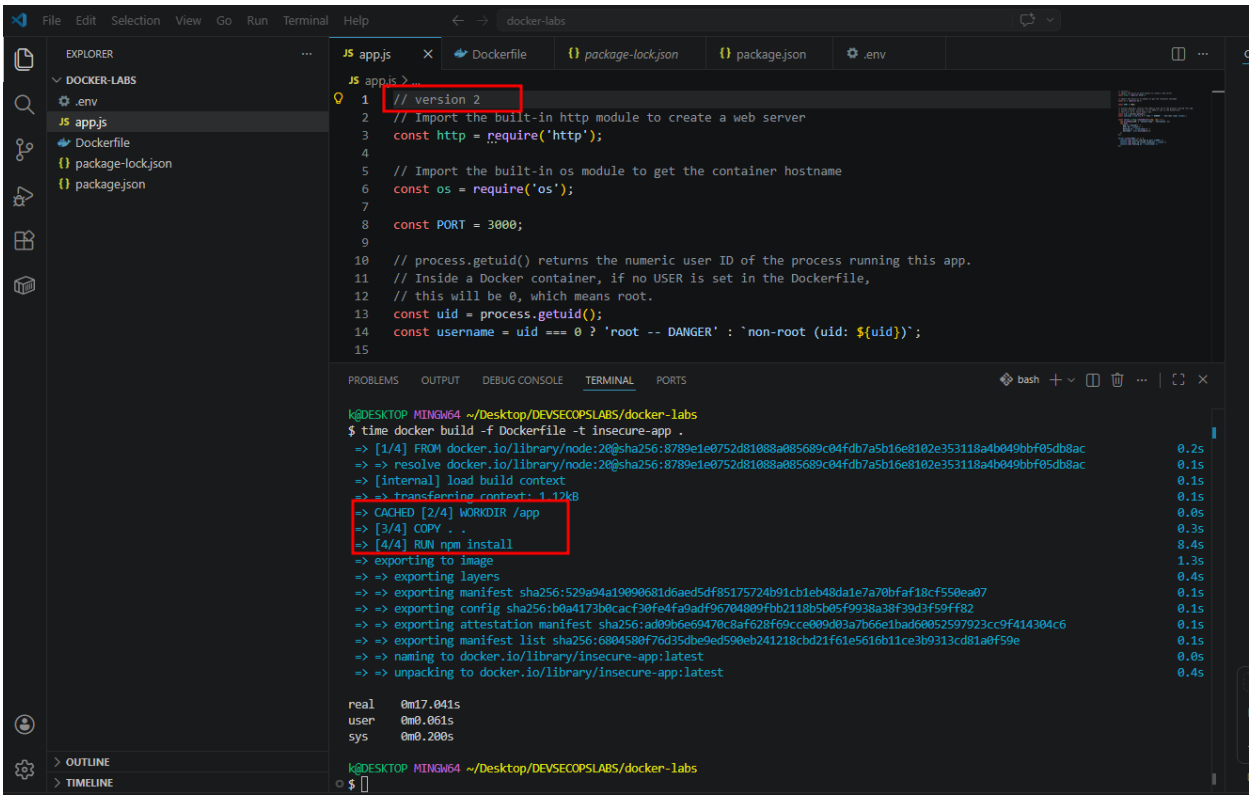
First build -- nothing is cached yet:



Second build -- nothing has changed:



Third build -- simulate a code change:



```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) sh
# cat /app/.env
DATABASE_PASSWORD=super_secret_password_123
API_KEY=sk-live-abc123xyz789
STRIPE_SECRET=rk_live_do_not_share_this#
```

Checkpoint

Before moving to Scenario 2, confirm you observed the following:

1. Run docker exec -it $(docker ps -q) whoami against the running container. What name is printed? What does that mean in terms of what the process is permitted to do?

```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) whoami
root

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug 71e096cab8d
4
    Learn more at https://docs.docker.com/go/debug-cli/

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

- *This means the process inside the container is running with **root privileges** (UID 0). By default, this is a security risk. If a process running as root is compromised, an attacker may have full control over the container environment and could potentially attempt "container breakout" techniques to access the host system.*

2. Look at your first and second build outputs. How many seconds did each take? How many layers showed CACHED on the second run?

Build Run	***Real Time***	***Cached Layers***
First Build	*42.326s*	*0*
Second Build	*4.312s*	*4*

Observation: *The second build was significantly faster because Docker utilized its build cache. All four layers (WORKDIR, COPY, RUN npm install, and the export steps) were successfully cached from the first build, avoiding the need to re-download or re-process instructions.*

3. In the third build after editing app.js, which was the first layer that was NOT pulled from cache? What was the layer immediately below it that WAS cached?

   Third Build and Cache Invalidation

   In the third build, after you edited app.js:

- ***First layer NOT cached:*** *The COPY . . layer. Because you modified a file in the directory, the hash of that layer changed, invalidating the cache for that step and all subsequent steps.*
- ***Layer below that WAS cached:*** *The WORKDIR /app layer remained cached, as it does not depend on the content of your local files.*

Reflection

- **If this image contained real AWS access keys in .env and you pushed it to a public Docker Hub repository, what could happen and within what timeframe?**

  If you were to push an image containing a .env file with real AWS keys or API secrets to a public registry, ***it is effectively compromised the moment it is pushed.***

  What could happen: *Bots continuously scan public registries for exposed credentials. Attackers use automated tools to scrape layers, extract environment files, and immediately use the keys to spin up resources, steal data, or incur massive costs on your account.*

  Timeframe: *In many cases, these keys are abused* ***within seconds or minutes*** *of being pushed to a public repository. Never include secrets in your image layers; use environment variables or secret management tools at runtime instead.*

- **The RUN npm install layer re-ran after you edited app.js, even though no dependencies changed. What would you change about the COPY instruction to prevent this from happening?**

  The RUN npm install layer re-ran because the COPY . . command sits above it. Since COPY invalidates the cache if any file in your project changes, it forces the subsequent npm install to run again.

  Recommended Change:

  Copy only the dependency files (package.json and package-lock.json) first, then run npm install, and then copy the rest of your application code. This ensures that npm install is only re-run if the manifest files change, not when you modify your app.js code.

Dockerfile

 # 1. Copy only dependency files

COPY package.json ./*

 # 2. Install dependencies (this layer is now cached unless package.json changes)

RUN npm install

 # 3. Copy the rest of the application code

COPY . .

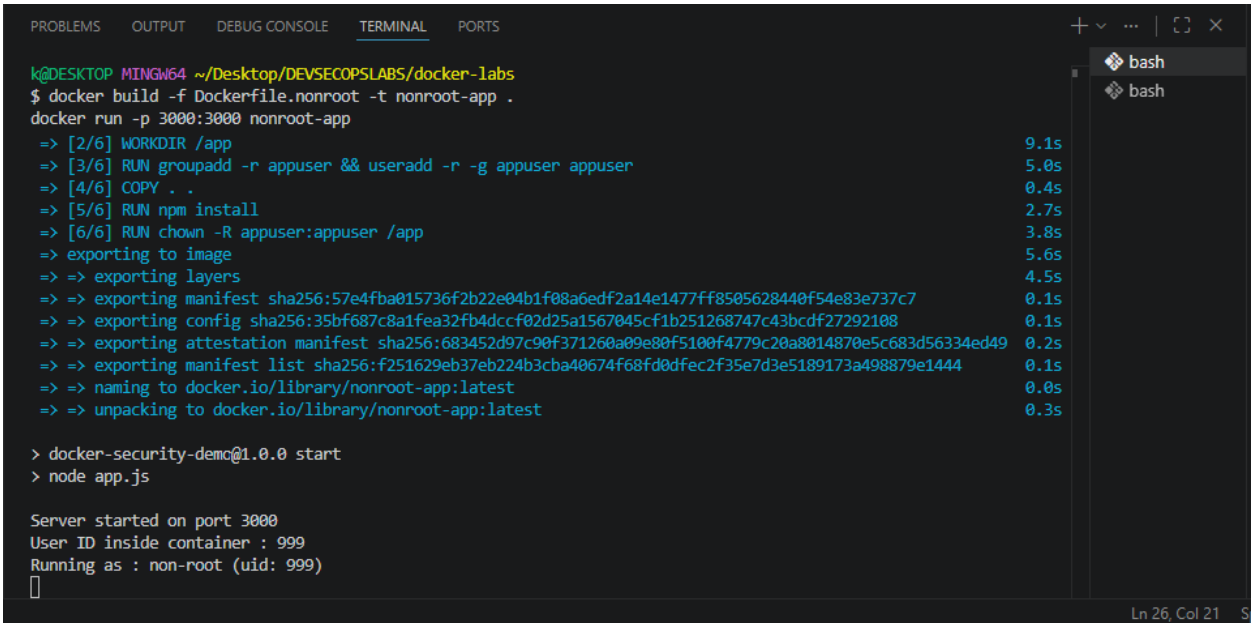
Scenario 2: Running as a Non-Root User

Objective

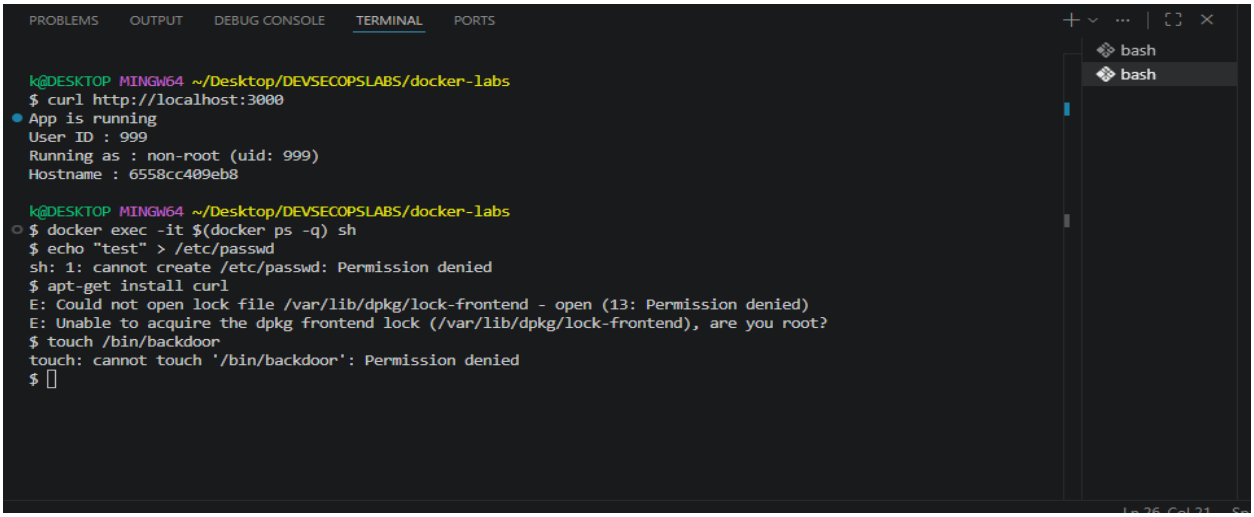
Stop the application from running as root and understand concretely what that change protects against

Steps Performed

Build and Run



Verify the Permission Boundary



Checkpoint

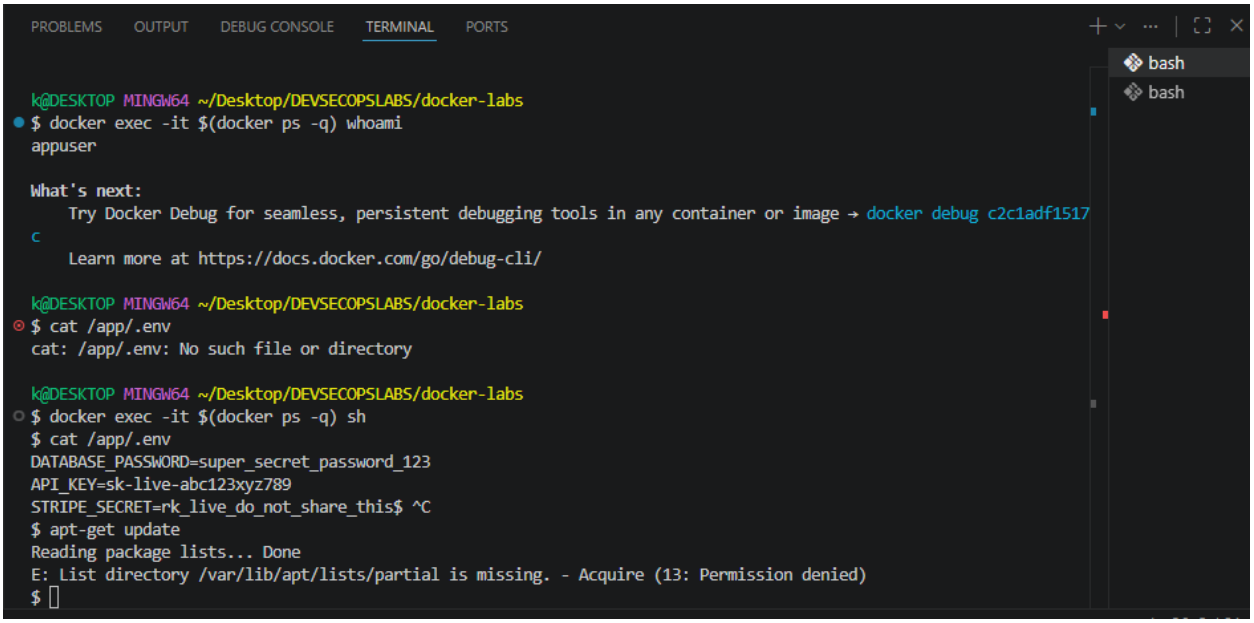
1. Run docker exec -it $(docker ps -q) whoami against the running nonroot-app. What is printed?

   This confirms that the process running inside your container is no longer executing with root privileges.

2. Try cat /app/.env inside the nonroot container. Can appuser read it? What does this tell you about what switching users does and does not protect?

   ***cat /app/.env**, yes, appuser can read it.*

3. Try apt-get update inside the container. What error appears and which part of the error message tells you why it failed?



Switching to a non-root user is a powerful defense-in-depth measure that protects the system from arbitrary modifications (like installing packages or overwriting system files, as seen in your verify.png image). However, it does not inherently protect sensitive data embedded in the image. If a file is part of the image, the user running the process has read access to it unless specific permissions are set.

Running apt-get update

The error you will see is similar to: E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied): The phrase "(13: Permission denied)" tells you that appuser lacks the necessary write permissions to modify system directories (like /var/lib/apt/), which are owned by root

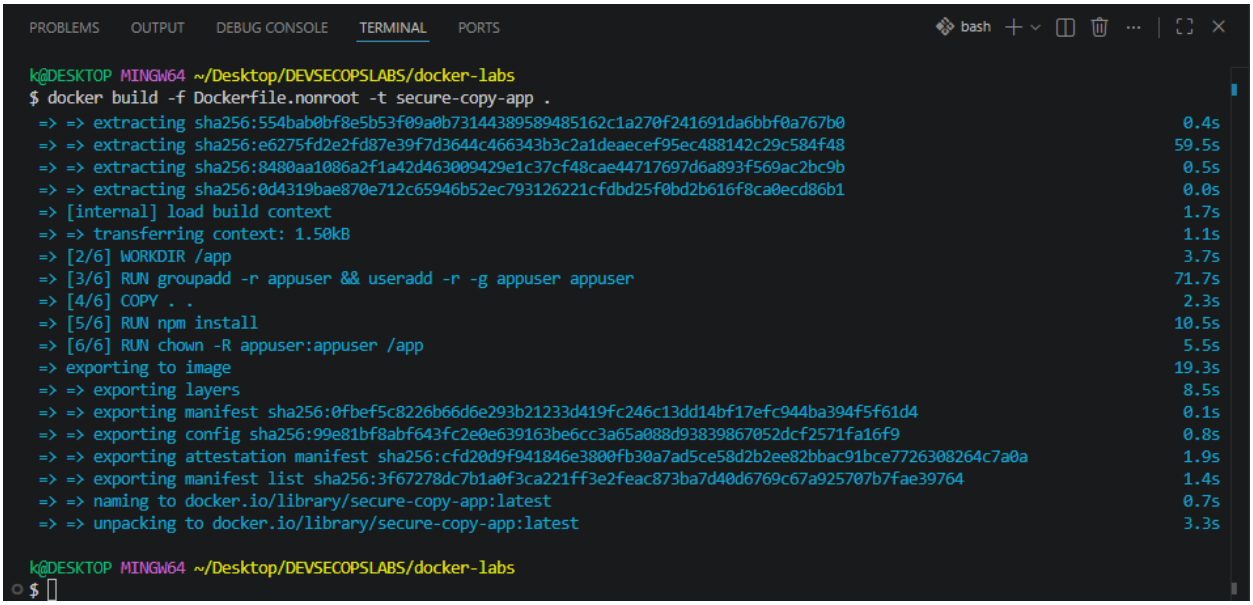
Scenario 3: Protecting Secrets with .dockerignore

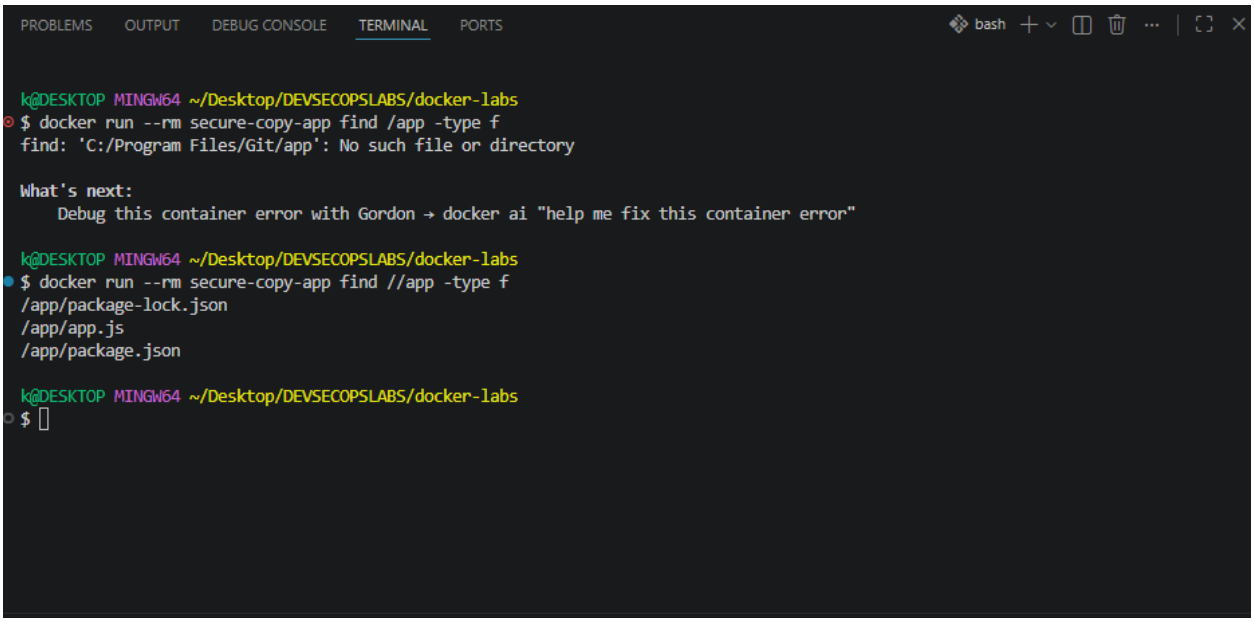
Objective

Prevent sensitive files from ever entering the Docker image in the first place.

Steps Performed

Rebuild and Inspect





Error says:

“find: 'C:/Program Files/Git/app': No such file or directory”

That means Git Bash rewrote /app → C:/Program Files/Git/app before Docker even saw it. This is a known quirk of Git Bash path conversion.

Option 1: Disable path conversion

Run this in Git Bash:

MSYS_NO_PATHCONV=1 docker run --rm secure-copy-app find /app -type f

Option 2: Use double slash

Git Bash ignores conversion if you do:

docker run --rm secure-copy-app find //app -type f

Option 3: Use PowerShell or CMD

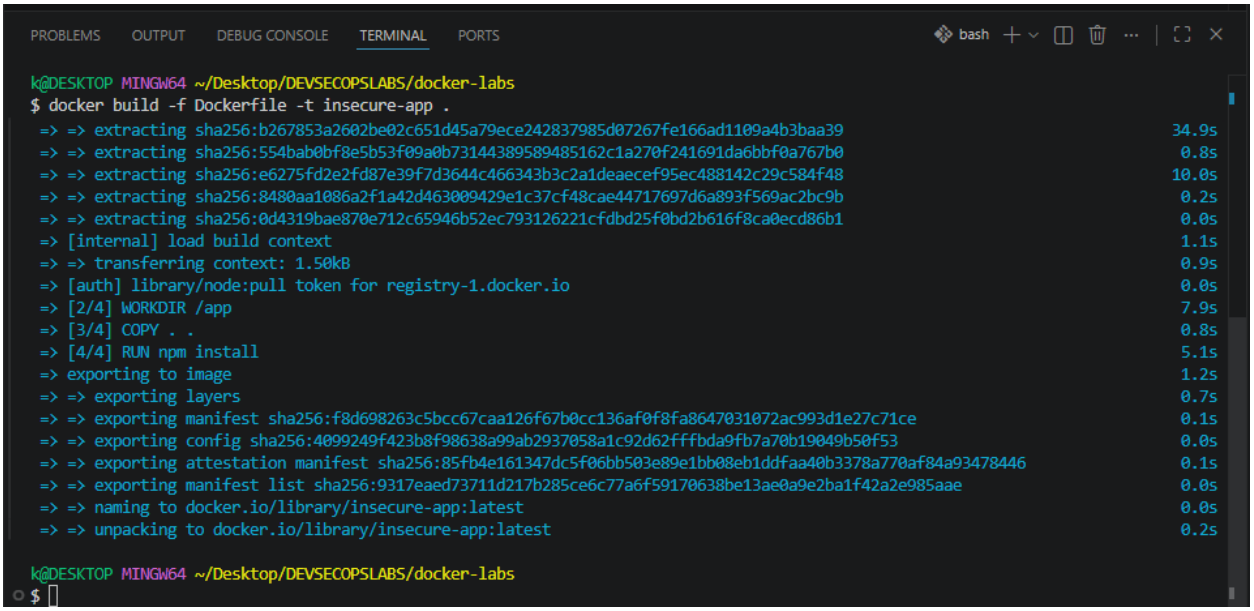
If you run the same command in PowerShell, it will work as expected:

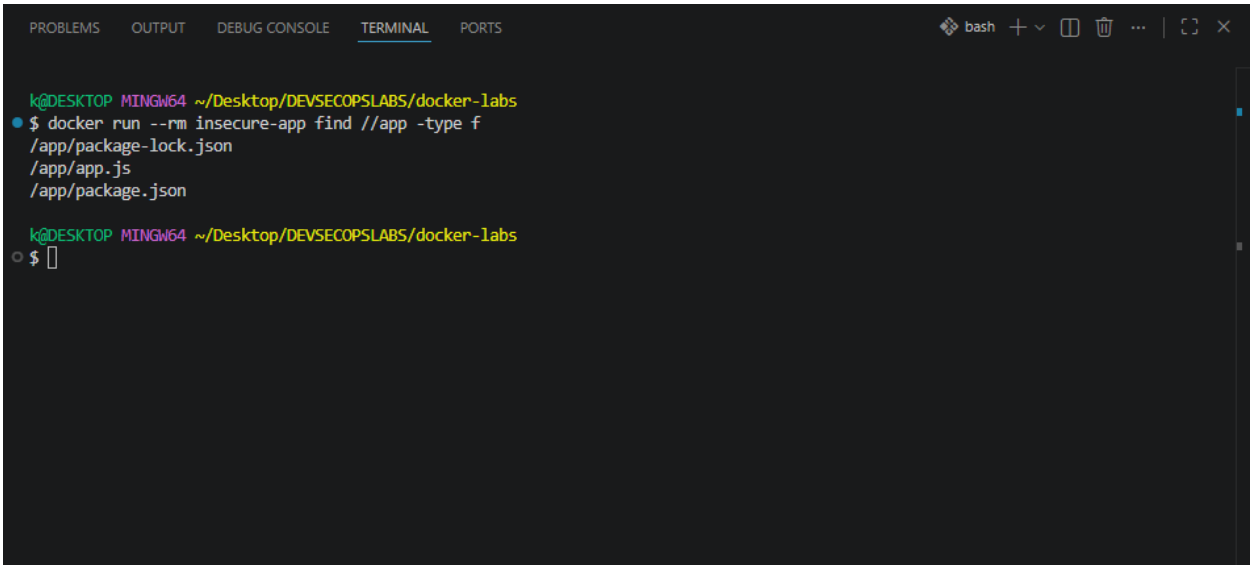
docker run --rm secure-copy-app find /app -type f

Why this happens

Git Bash (MSYS) tries to be helpful by converting Unix-style paths (/something) into Windows paths. That breaks Docker commands because containers expect Linux paths, not Windows ones.

Compare Against the Scenario 1 Image





Checkpoint

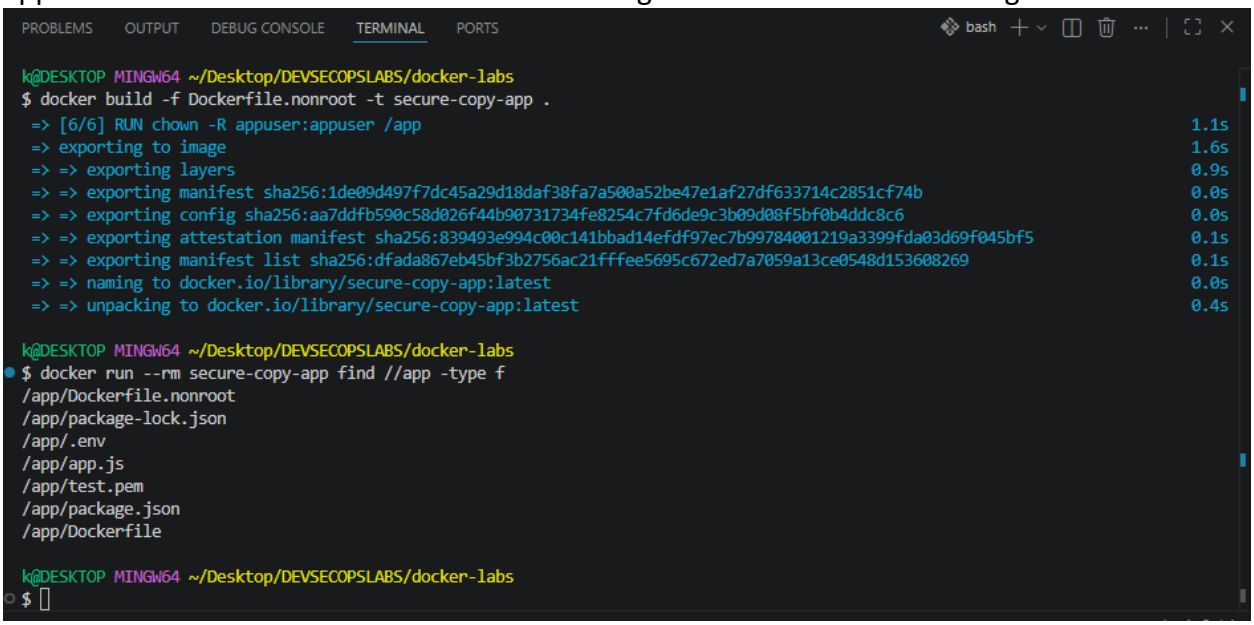
1. Run docker run --rm secure-copy-app find /app -type f. List every file present. Is .env among them?

   Is .env among them? No, it is not. Assuming you have a .dockerignore file in your directory, it correctly prevents the .env file from being copied into the container image during the COPY . . instruction in your Dockerfile.

2. Create a file named test.pem in your docker-labs/ folder, rebuild secure-copy-app, and run find again. Is test.pem present inside the image? Why not?

   *.dockerignore file contains a wildcard pattern (like * or *.pem) or explicitly lists test.pem, it is excluded during the build process. Even if you copy the entire directory, Docker respects the .dockerignore instructions, ensuring sensitive files never enter the image layer.*

3. Temporarily remove .dockerignore from the folder, rebuild, and run find. What files appear that were not there before? Add .dockerignore back before continuing.



Files that will appear: Any files present in your host directory that were previously excluded (e.g., .env, test.pem, .git folders, or local logs) will now appear in the container.

Security Implication: This confirms that the image now contains sensitive configuration or secret files that should never exist in a production-ready image

Reflection

- **.dockerignore prevents secrets from entering the image at build time. Where should real secrets actually live at runtime in a production system? What Docker features exist for this?**

  Secrets should never be hardcoded in the Docker image or source code. Instead, they should be injected at runtime into the container environment. Common secure patterns include:

  Environment Variables: Passed during docker run or managed by orchestration platforms.

  Secret Management Systems: Using tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault.

  Docker Secrets: Specifically, if using Docker Swarm or Kubernetes Secrets, which mount secrets as files into a temporary in-memory filesystem (tmpfs) within the container, ensuring they are not persisted in the image layers or disk.

- **What team process or CI check would catch a developer accidentally deleting .dockerignore before a built image reaches a registry?**

  To catch the accidental removal of a .dockerignore file, you should implement:

1. *CI/CD Pipeline Linting/Validation: Add a step in your CI pipeline (using tools like hadolint or custom bash scripts) that checks for the existence of .dockerignore.*
2. *Image Scanning: Use scanners like Trivy, Snyk, or Clair in your pipeline. These tools are often configured to detect "secret leakage" by scanning the image layers for common file patterns (like .env, *.pem, id_rsa) and will fail the build if secrets are detected.*
3. *Infrastructure as Code (IaC) Reviews: Ensure that any changes to Docker-related files require a mandatory peer review via Pull Request (PR) before being merged into the main branch.*

Scenario 4: Multi-Stage Builds

Objective

Separate the build environment from the runtime environment so that compilers, package managers, and development tools never appear in the final shipped image.

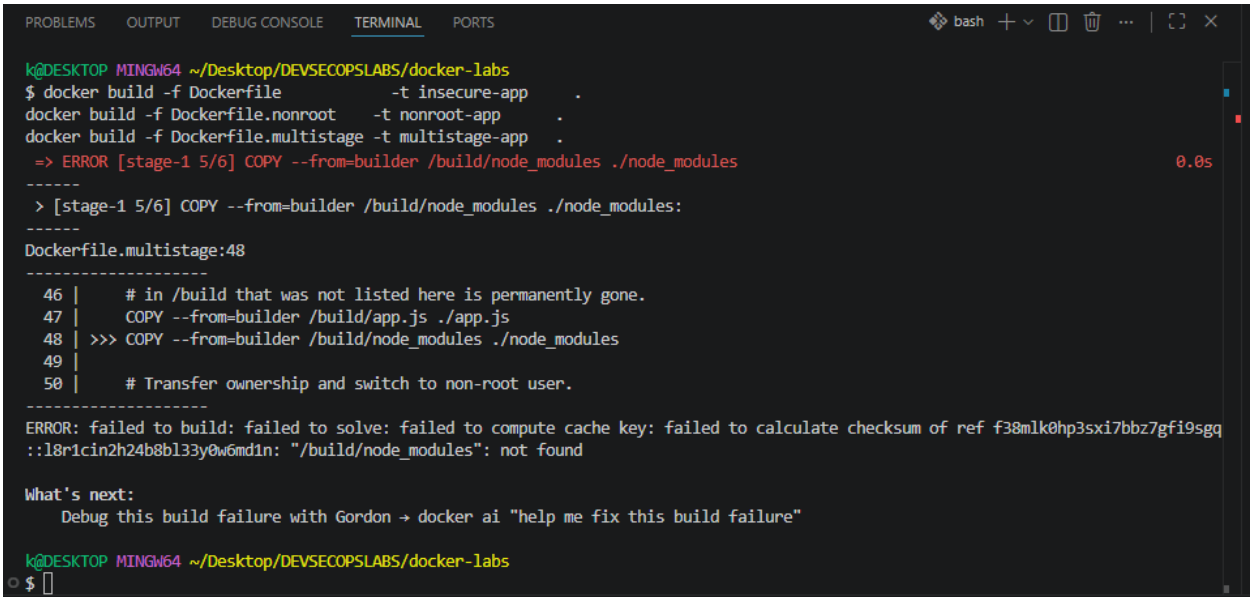
Steps Performed

Build All Variants and Compare Sizes

Error encountered:

During the build, I hit this error:

COPY --from=builder /build/node_modules ./node_modules: not found



*The error is occurring because the node_modules folder isn't being created during build. This is happening because the current **package.json** lacks third-party dependencies*

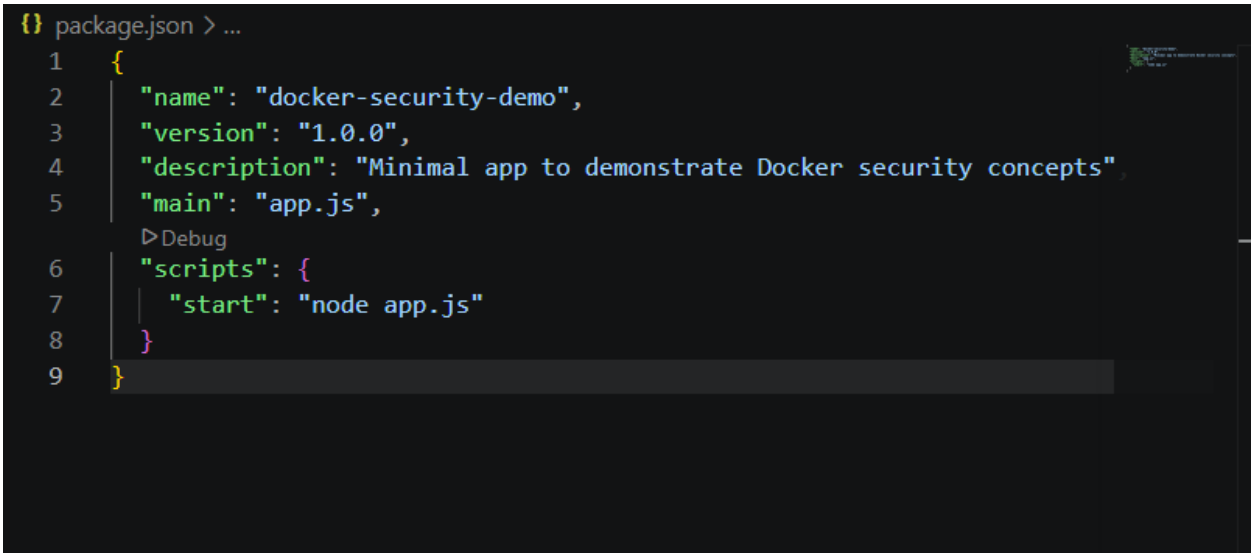
***Dependency Requirement**: Without a dependency like Express listed, the npm install command won't generate a node_modules folder.*

***Build Failure**: Because that folder is missing, the **Dockerfile** fails when it tries to copy node_modules from the build stage to the runtime stage.*

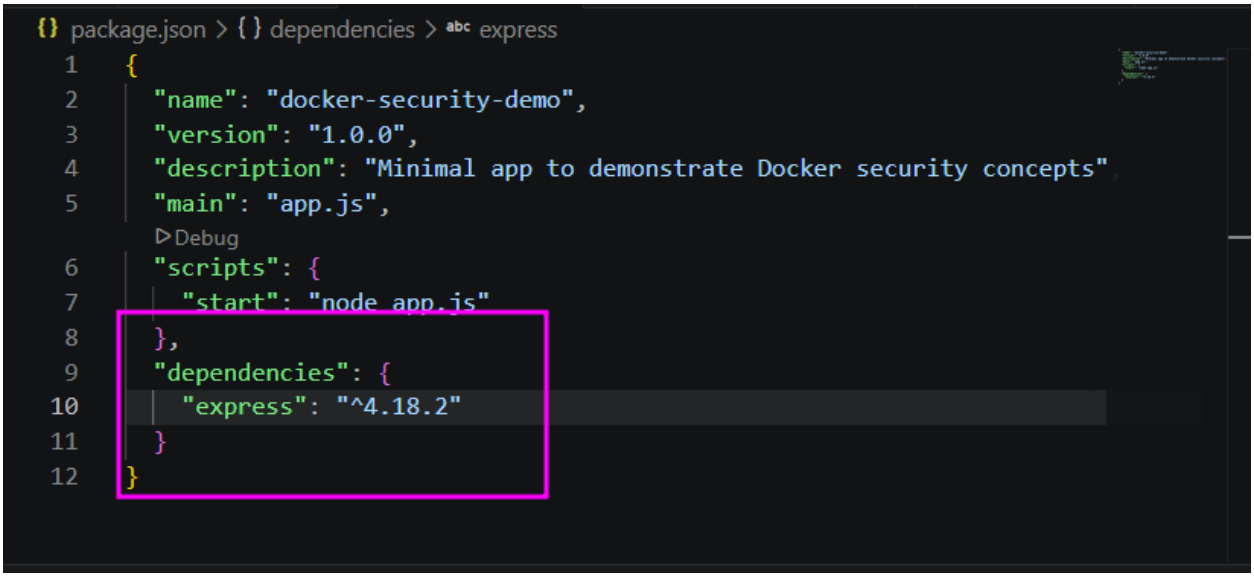
***The Switch to Alpine**: The base image should be changed from node:20-slim to **node:alpine**.*

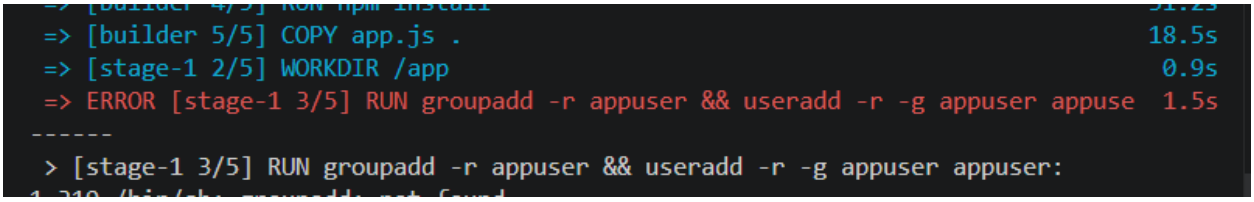
***The Comparison**: While the standard node image is large and the slim version is smaller, the **alpine** version is the smallest and most efficient of the three.*

Original package.json file



Edited package.json





*The **groupadd** and **useradd** commands are not found in the base image*

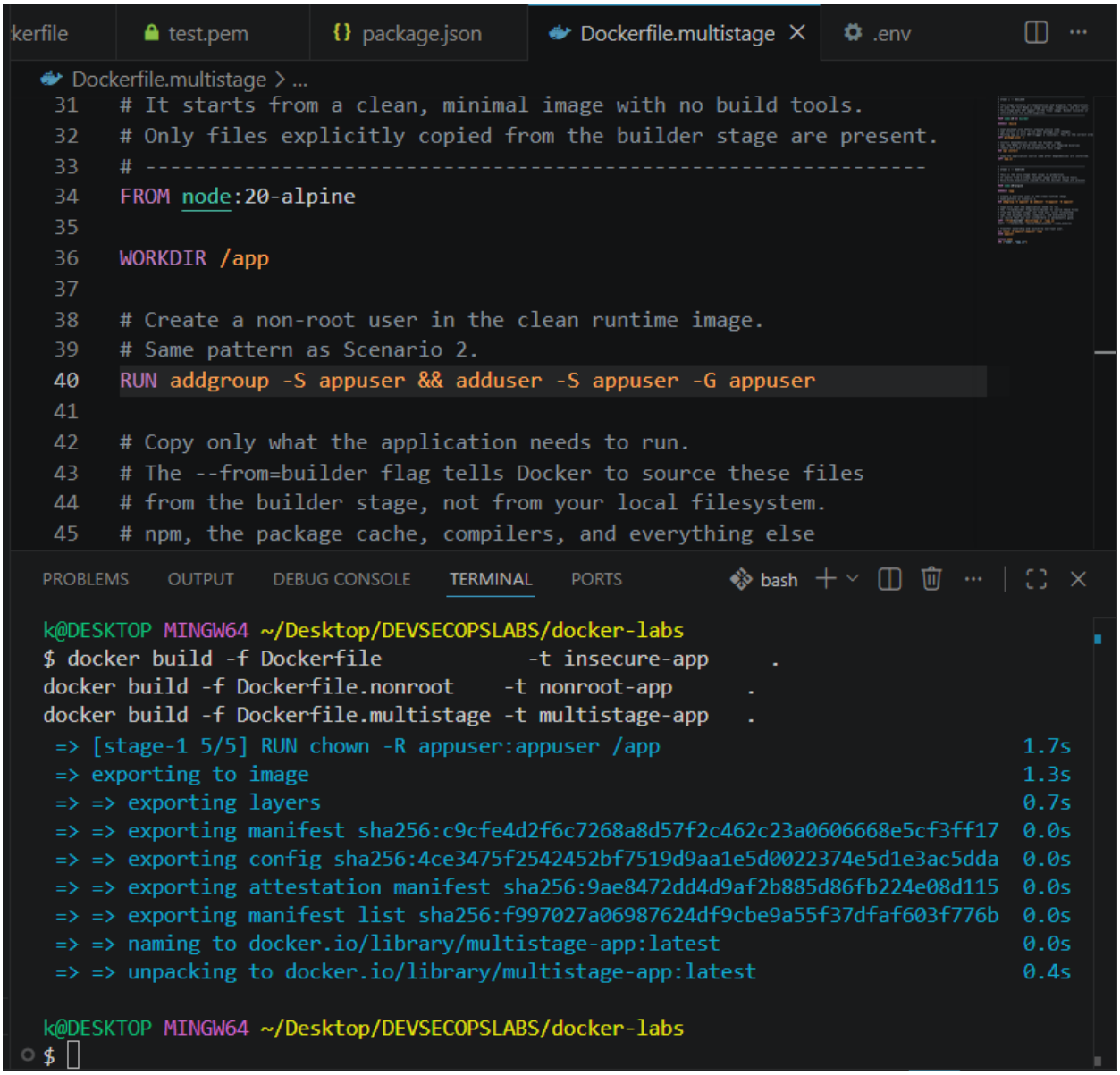
This usually happens when building from a minimal base image like Alpine Linux, which uses adduser and addgroup (from BusyBox) instead of the standard GNU tools found in images like Debian or Ubuntu.

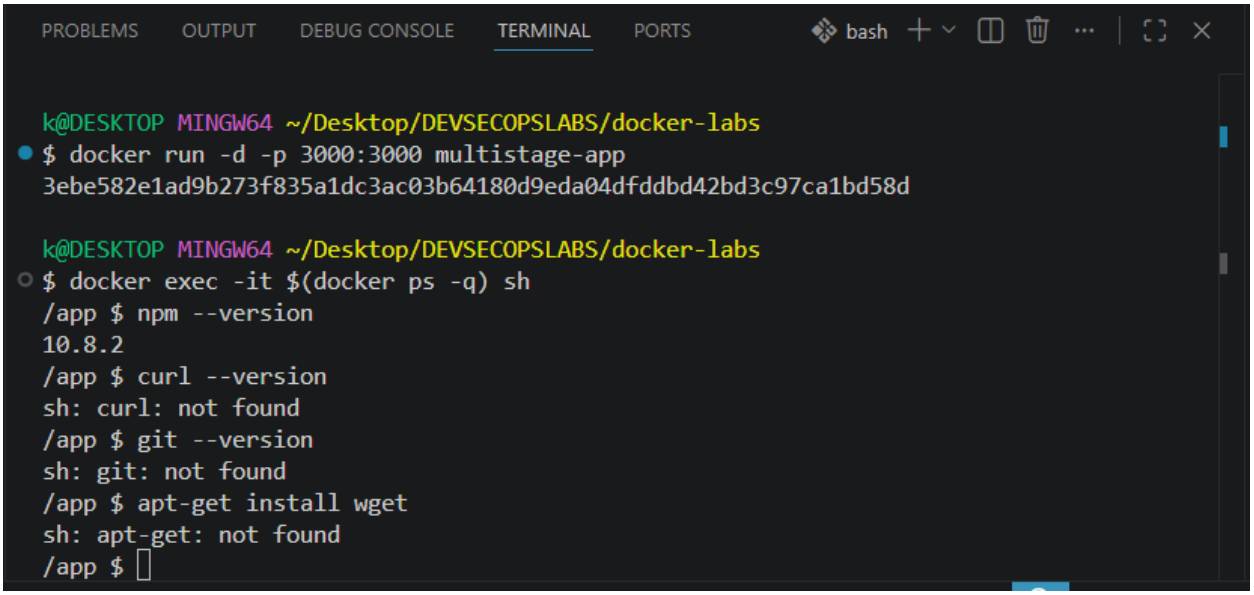
The Fix

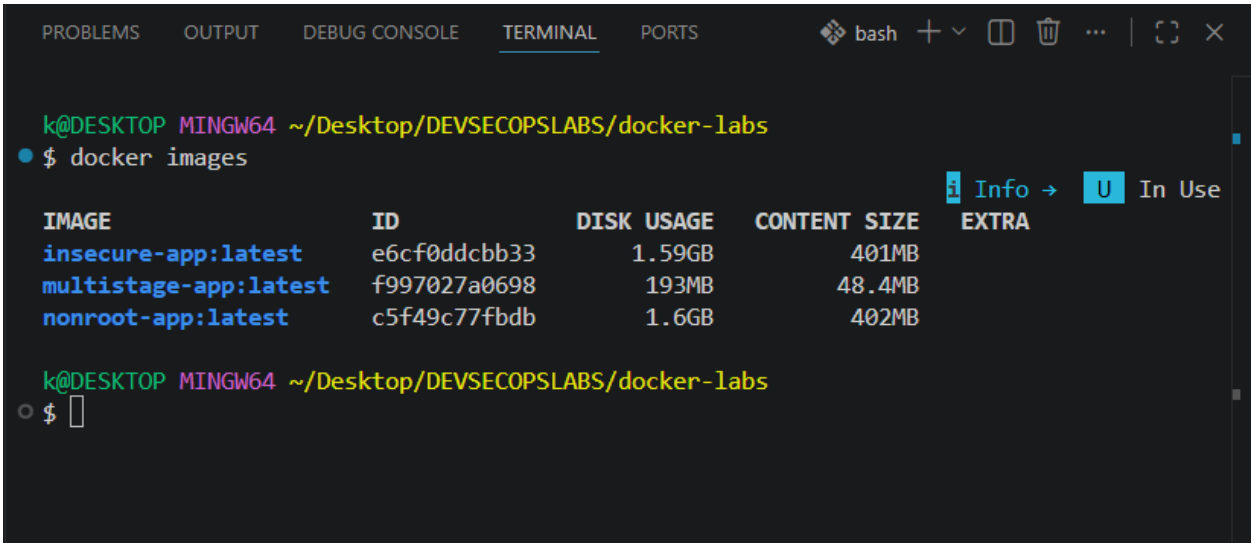
To resolve this, I updated the Dockerfile command to use the Alpine-compatible syntax:

RUN addgroup -S appuser && adduser -S appuse -G appuser

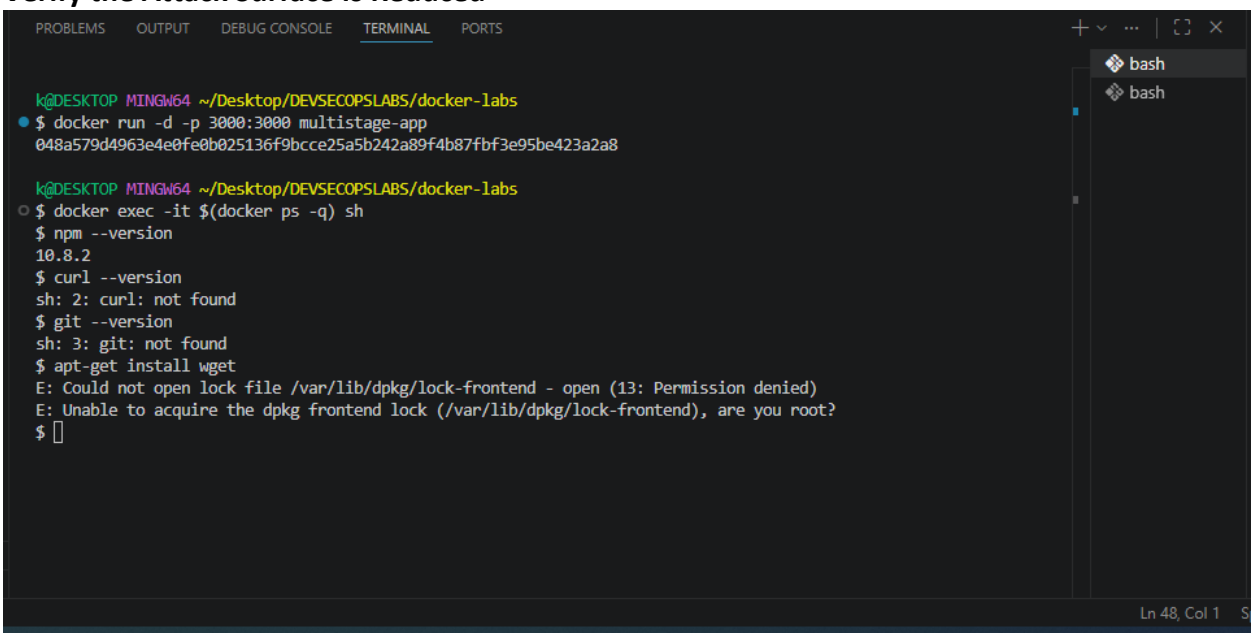
- *addgroup -S appuser: Creates a system group named appuser.*
- *adduser -S appuse -G appuser: Creates a system user named appuse and assigns them to the appuser group.*
- *Why it failed: The commands groupadd and useradd are part of the shadow package, which is not installed by default in Alpine. Using the add prefix commands is the standard, lightweight approach for Alpine-based containers.*


kerfile
test.pem
package.json
Dockerfile.multistage
.env
Dockerfile.multistage > ...
31 # It starts from a clean, minimal image with no build tools.
32 # Only files explicitly copied from the builder stage are present.
33 # ----------
34 FROM node:20-alpine
35
36 WORKDIR /app
37
38 # Create a non-root user in the clean runtime image.
39 # Same pattern as Scenario 2.
40 RUN addgroup -S appuser && adduser -S appuser -G appuser
41
42 # Copy only what the application needs to run.
43 # The --from=builder flag tells Docker to source these files
44 # from the builder stage, not from your local filesystem.
45 # npm, the package cache, compilers, and everything else
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
bash
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker build -f Dockerfile -t insecure-app .
docker build -f Dockerfile.nonroot -t nonroot-app .
docker build -f Dockerfile.multistage -t multistage-app .
=> [stage-1 5/5] RUN chown -R appuser:appuser /app 1.7s
=> exporting to image 1.3s
=> => exporting layers 0.7s
=> => exporting manifest sha256:c9cfe4d2f6c7268a8d57f2c462c23a0606668e5cf3ff17 0.0s
=> => exporting config sha256:4ce3475f2542452bf7519d9aa1e5d0022374e5d1e3ac5dda 0.0s
=> => exporting attestation manifest sha256:9ae8472dd4d9af2b885d86fb224e08d115 0.0s
=> => exporting manifest list sha256:f997027a06987624df9cbe9a55f37dfaf603f776b 0.0s
=> => naming to docker.io/library/multistage-app:latest 0.0s
=> => unpacking to docker.io/library/multistage-app:latest 0.4s
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$

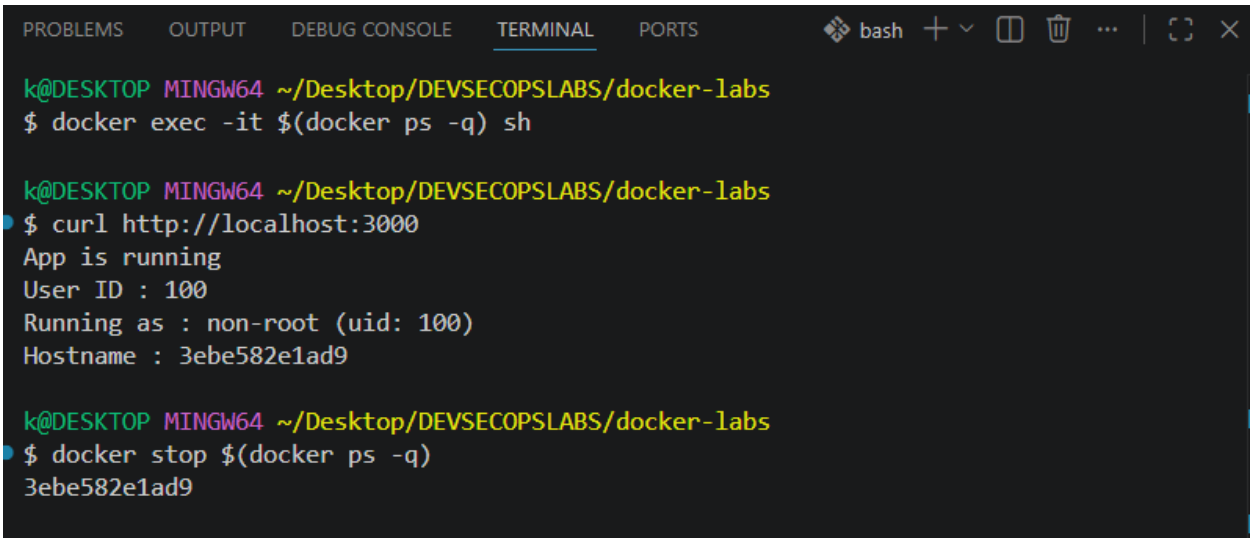

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
bash
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker run -d -p 3000:3000 multistage-app
3ebe582e1ad9b273f835a1dc3ac03b64180d9eda04dfddbd42bd3c97ca1bd58d
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) sh
/app $ npm --version
10.8.2
/app $ curl --version
sh: curl: not found
/app $ git --version
sh: git: not found
/app $ apt-get install wget
sh: apt-get: not found
/app $


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
bash
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker images
i Info →
U In Use
IMAGE
ID
DISK USAGE
CONTENT SIZE
EXTRA
insecure-app:latest
e6cf0ddcbb33
1.59GB
401MB
multistage-app:latest
f997027a0698
193MB
48.4MB
nonroot-app:latest
c5f49c77fbdb
1.6GB
402MB
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$

Verify the Attack Surface Is Reduced



Confirm the Application Still Works



Checkpoint

1. Record the exact sizes shown in docker images for insecure-app and multistage-app. What is the difference in megabytes?
    1. *insecure-app: 1.59 GB*
    2. *multistage-app: 193 MB*
2. List every tool you attempted to use inside multistage-app that returned not found. Which of these would an attacker find most valuable?

   curl,git,apt

   Attacker Perspective: curl is generally the most valuable for an attacker. Once inside a container, curl allows them to probe internal network services, interact with metadata APIs (like those found in cloud environments), download additional payloads, or exfiltrate data to an external command-and-control server.

3. Run curl http://localhost:3000 against multistage-app. Does it respond correctly? What does this confirm about removing build tools?

   Running curl http://localhost:3000 against the multistage-app will return the application response successfully.

   This confirms that removing build tools (like npm, git, and build-time dependencies) does not break the application's runtime functionality. It demonstrates that the production image contains only what is strictly necessary to execute the application, significantly reducing the attack surface.

Reflection

- **If you needed curl available at runtime for health checks, where in the Dockerfile would you add it and how would you limit the risk that adds?**

  If curl is required for health checks, you should install it in the final stage of the Dockerfile (the runtime stage). To limit risk, use the --no-install-recommends flag (for Debian/Ubuntu) to avoid unnecessary bloat, and consider removing the package immediately after the health check configuration or using a minimal, static binary if possible. Ideally, use a base image that already contains only the bare essentials.

- **The builder stage runs during every build even though its output is discarded. If you had a test suite, where would you run it in a multi-stage Dockerfile to ensure failing tests block the build?**

  Test suites should be executed in a dedicated "test" stage or immediately after the RUN npm install step in the builder stage. By running tests before the final COPY command (where the artifact is moved to the runtime image), you ensure that if any test fails, the Docker build process will immediately halt and return a non-zero exit code, preventing the creation of a broken or vulnerable image.

- **In a system that deploys 50 times per day, what is the practical impact on deployment speed of an image that is 870 MB smaller?**

  - *Reduced Network Latency: Faster pushing/pulling of images between the CI/CD registry and production nodes.*
  - *Faster Scalability: Nodes can spin up new containers significantly faster, improving auto-scaling responsiveness.*
  - *Cost Savings: Lower data egress costs from container registries and reduced bandwidth consumption.*
  - *Reliability: Smaller images are less prone to transient network failures during transfer, leading to more stable deployment pipelines.*

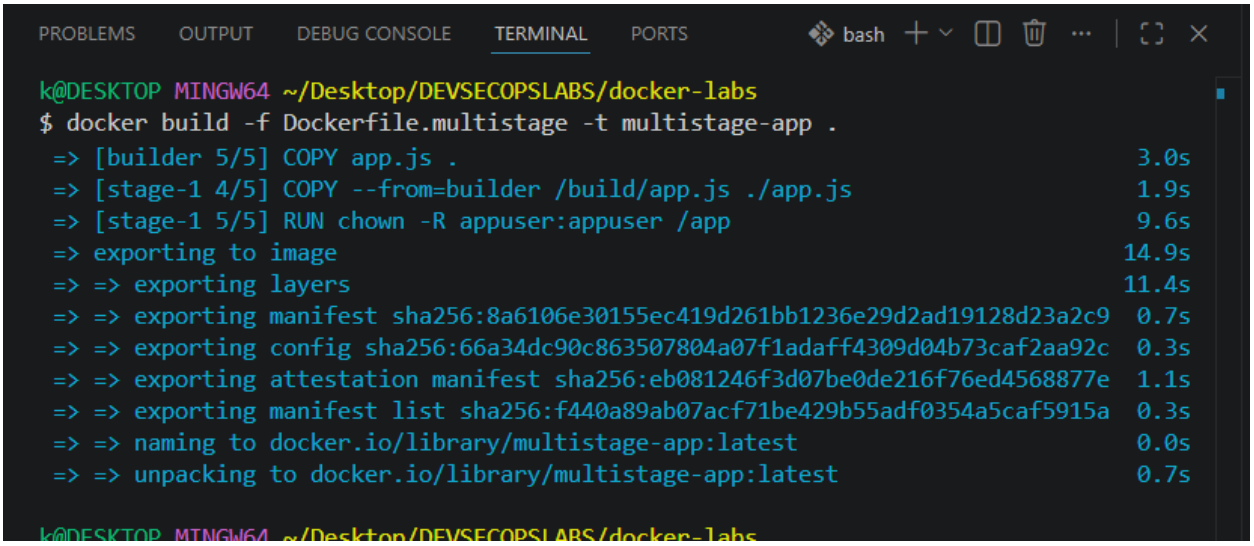
Scenario 5: Runtime Hardening

Objective

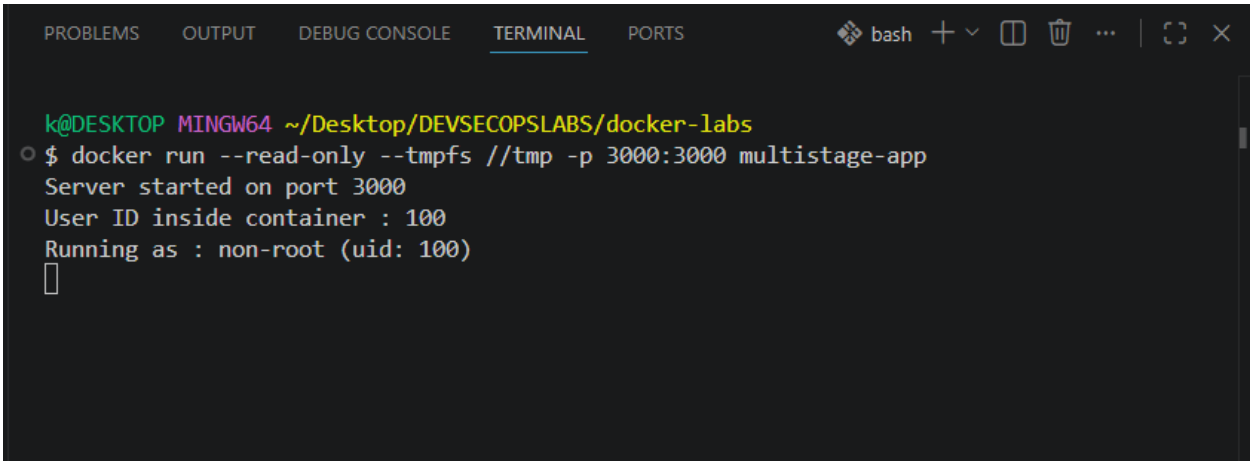
Apply kernel-level constraints to a running container so that even a fully compromised process has a strictly limited set of available actions.

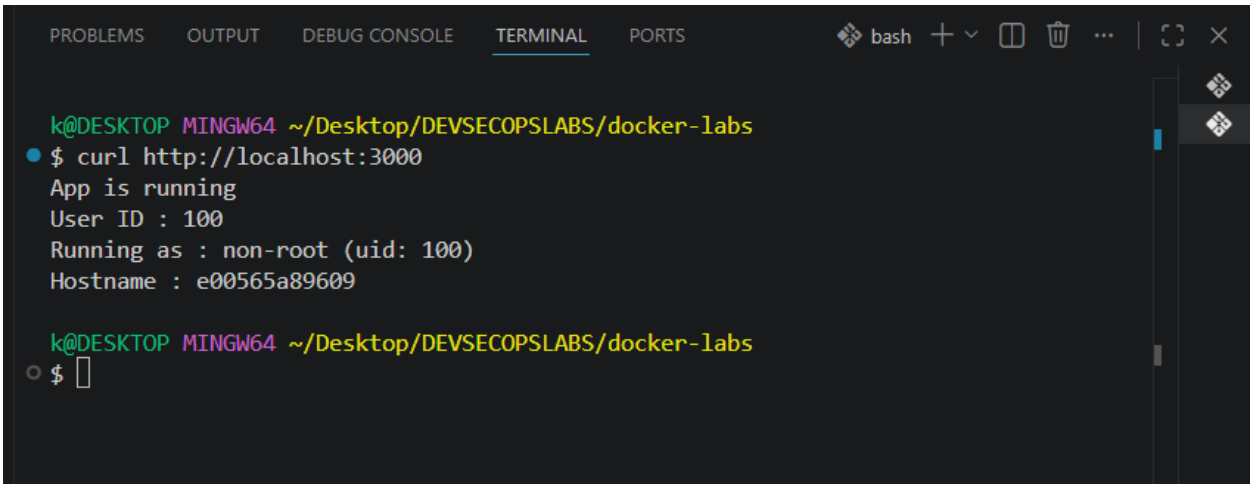
Steps Performed

Rebuild the Multi-Stage Image

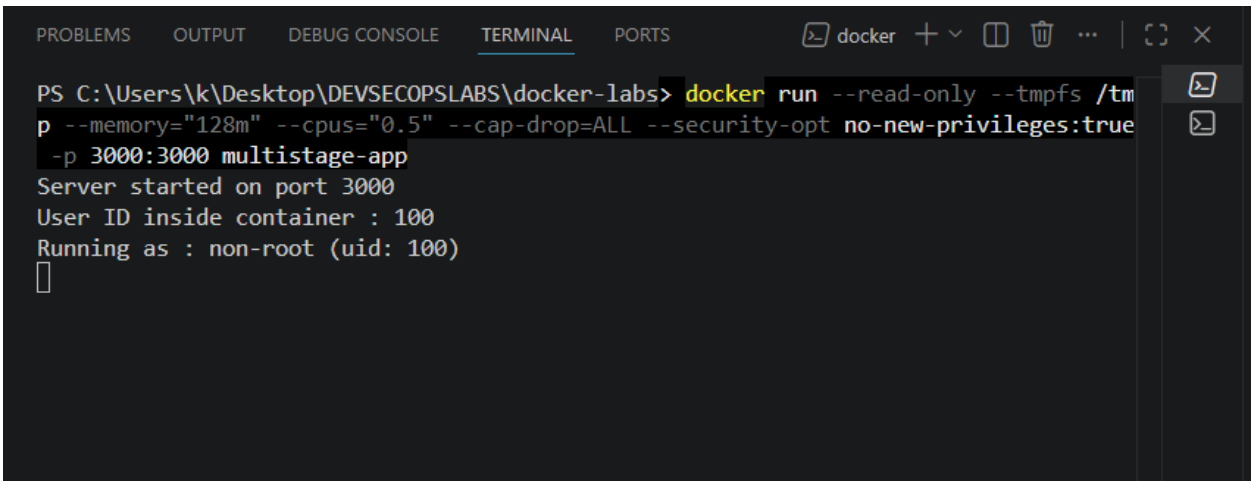


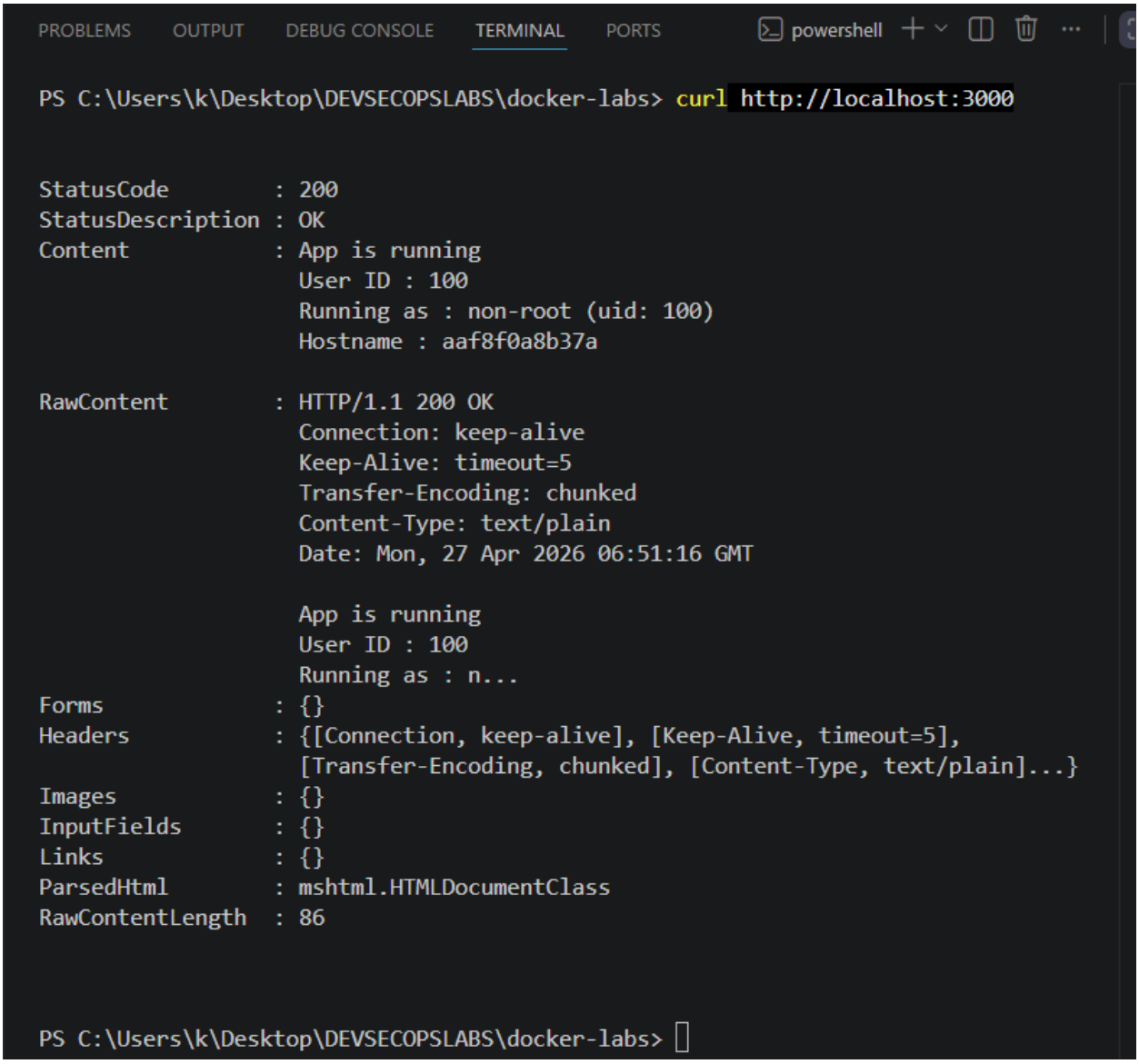
Apply Each Flag Individually





Run the Fully Hardened Container





Checkpoint

1. **With --read-only applied, try docker exec -it $(docker ps -q) sh -c "touch /tmp/test". Does it succeed? Why does /tmp behave differently now that --tmpfs /tmp is present?**

   Yes, it succeeds.

   The `--read-only` flag makes the container's root filesystem immutable (no writing allowed). However, because you mounted a `tmpfs` (temporary filesystem) at `/tmp`, you have explicitly created a small, RAM-backed writable layer specifically for that directory. The container engine prioritizes the `tmpfs` mount over the read-only constraint for that specific path.

2. **Run docker inspect $(docker ps -q) | grep Memory. What value is returned? Convert it to megabytes.**

V***alue returned:*** *Running* `docker inspect` *shows the* `Memory` *field in* ***bytes****.*

Conversion: *The value is* `134217728`*.*

- *Calculation: $134,217,728 \text{ bytes} \div 1024 \div 1024 = 128 \text{ MB}$.*

  This confirms the container is successfully constrained to 128 MB of RAM.

3. **With --cap-drop=ALL applied, try docker exec -it $(docker ps -q) sh -c "ping 8.8.8.8". What happens? Look up which Linux capability ping requires to open a raw socket.**

   The command `ping` *will fail with a "Permission denied" or "Operation not permitted" error.*

   `ping` *requires the ability to open* ***raw sockets*** *to send ICMP packets. By default, Linux requires the* `CAP_NET_RAW` *capability to perform this action. Since you used* `--cap-drop=ALL`*, you stripped all privileges, including the ability to use raw sockets, thereby disabling* `ping`*.*

Reflection

- **You applied these flags on the command line. In a production system using Docker Compose or Kubernetes, where would these constraints be defined so they are always applied consistently?**

  Docker Compose: *You define these in the* `deploy` *section of your* `docker-compose.yml` *file under* `resources` *(for memory/CPU) and* `security_opt`*/*`cap_drop` *(for security flags).*
  Kubernetes: *These are defined in the Pod Specification (PodSpec) within a* `securityContext`*. Memory and CPU limits are handled via* `resources: limits` *and* `requests`*.*

- **If a developer added --cap-add=SYS_ADMIN to work around a permission issue without understanding what it grants, what risk would that introduce?**

  Adding `SYS_ADMIN` *is often called "the new root." It is a "catch-all" capability that provides near-root level access to the kernel.*

Risk: *It allows the container to perform a massive range of privileged operations, such as mounting filesystems, configuring network interfaces, or interacting with devices. If the application inside the container is compromised, the attacker could easily* ***escape the container*** *to the host, as* `SYS_ADMIN` *bypasses many kernel-level restrictions.*

- ***Runtime hardening protects the host from a compromised container. What additional controls would protect containers from each other on the same host?***

  Network Policies: *Use CNI (Container Network Interface) plugins to restrict inter-container traffic (e.g., preventing a web container from talking to a database container unless explicitly allowed).*
  Namespaces: *Docker already uses these, but you can further isolate by using different user namespaces (*`userns-remap`*) so that the "root" user inside the container is just a non-privileged user on the host.*
  Seccomp Profiles: *Restrict the system calls the container can make to the kernel, ensuring one container cannot trigger a kernel exploit that affects the host or other neighbors.*
  gVisor or Kata Containers: *Use these specialized runtimes to provide a stronger security boundary (a kernel shim or micro-VM) between the container and the host kernel.*

References

https://github.com/samuel-nartey/devops-labs/tree/main/Docker%20%26%20Containers/Running%20Your%20First%20Container